

Forecasting German Electricity Demand

A Comparative Study of Statistical, Machine-Learning and Deep-Learning Approaches

Ram Sai Sayani – 24080366 / MSc Data Science — Advanced Research Topics, Assignment 1

GitHub Link: <https://github.com/Ram-sai7/electricity-demand-forecasting.git>

1. Introduction

Grid operators and utilities need demand forecasts they can trust, covering everything from a few hours ahead to several years out, because scheduling generation, securing reserves, and planning long-term capacity all depend on a realistic picture of how much load the system will be asked to carry. This report builds and compares five families of forecasting model — simple statistical benchmarks, a seasonal ARIMA model, a temperature-conditioned SARIMAX model, tree-based ensemble regressors, and a recurrent neural network — on German national electricity load drawn from the Open Power System Data (OPSD) time-series package (Open Power System Data, 2020), spanning hourly load from 1 January 2015 through to the end of the available file in October 2020. Weekly mean temperature for Berlin, used here as a stand-in for Germany as a whole, came from the Open-Meteo historical archive (Open-Meteo, 2024) and served as an explanatory covariate in Part 4.

The question driving the analysis is not merely which model comes out with the lowest error, but why each one behaves the way it does given the choices behind it — the differencing orders settled on for SARIMA, the way lag features were built for the tree ensembles, and the sequence length and rollout strategy used for the LSTM. Section 2 covers the exploratory work and formal stationarity testing that shaped the SARIMA specification. Section 3 outlines each modelling approach and the reasoning for including it. Section 4 presents results over a held-out two-year (104-week) forecast horizon running from October 2018 to October 2020. Section 5 discusses, critically, why the models behaved as they did, including two outcomes that went against my initial expectations. Section 6 answers the set questions directly, Section 7 covers limitations and future work, and Section 8 concludes.

2. Data and Exploratory Analysis

The raw OPSD file gives hourly German load (DE_load_actual_entsoe_transparency) in megawatts. Restricting the series to 1 January 2015 onward leaves a cleaned hourly dataset of 50,400 observations running from 2015-01-01 to 2020-09-30. This was aggregated into a daily series (2,100 observations) and a weekly series (301 observations, converted to gigawatts for readability) by simple mean resampling, with any small gaps this created closed via time-based linear interpolation.

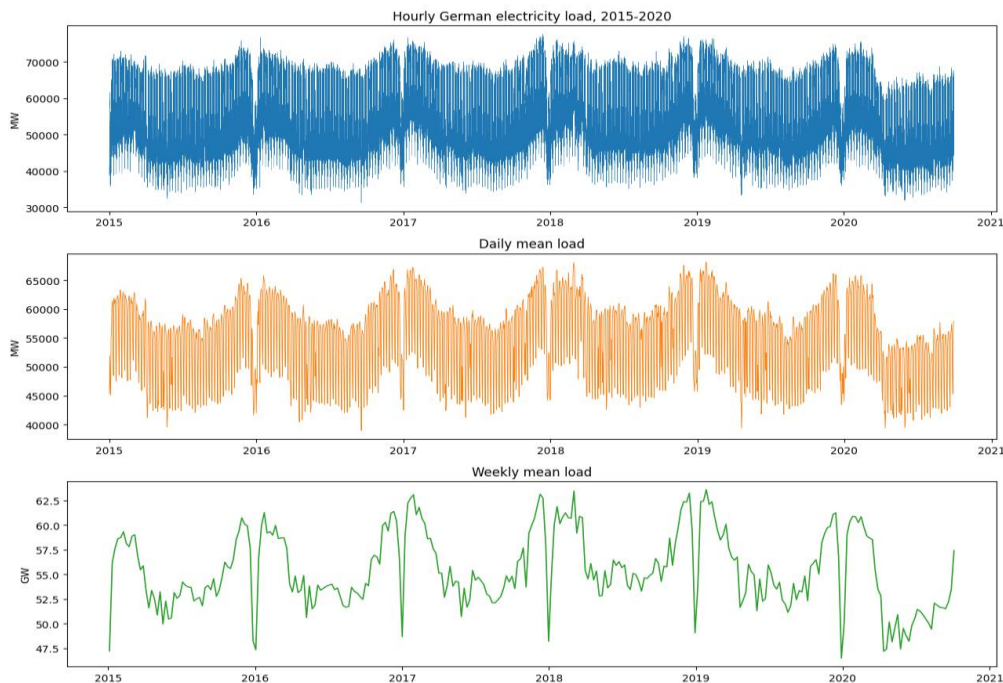


Figure 1. German electricity load at hourly, daily and weekly resolution, 2015–2020.

Figure 1 shows roughly what you'd expect at each resolution: intraday cycling dominates the hourly series but disappears once the data is averaged down to daily or weekly values, while the weekly series brings out the underlying annual pattern much more clearly — higher load in winter, with a secondary dip around the summer holiday period. A visible, somewhat irregular dip shows up toward the end of the series in 2020, matching the demand shock associated with COVID-19 lockdown measures. This looks like a genuine structural break rather than a data artefact, and its implications for the forecast evaluation are picked up again in Section 5.

2.1 Seasonal decomposition

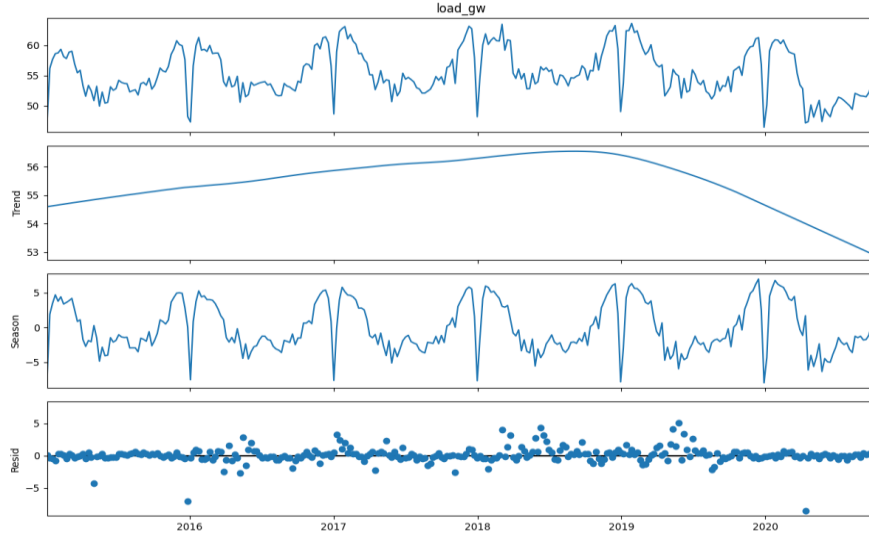


Figure 2. STL decomposition of the weekly load series (annual period, $s = 52$).

An STL decomposition (Cleveland et al., 1990) of the weekly series with an annual period returns a seasonal strength of 0.871 against a trend strength of just 0.383. In plain terms, the annual cycle accounts for most of the systematic movement in weekly load, while the trend component is comparatively weak and, as Figure 2 shows, not monotonic — it drifts rather than growing or shrinking consistently. This figure was the main reason for treating the problem as seasonal throughout: it justified fitting SARIMA rather than plain ARIMA, and giving the tree ensembles an explicit lag-52 feature instead of leaving them to find the annual cycle on their own.

2.2 Stationarity testing

Two complementary tests were run, since they test opposite null hypotheses: the Augmented Dickey-Fuller (ADF) test, whose null is that the series contains a unit root, and the KPSS test, whose null is that the series is stationary. Both were applied to the level series, the first difference, the seasonal (lag-52) difference, and the combination of the two, following the approach set out in Box et al. (2015) and Hyndman and Athanasopoulos (2021).

Series	ADF stat	ADF p	KPSS stat	KPSS p	Verdict
Level	-4.047	0.0012	0.160	0.10	Ambiguous (see discussion)
First difference ($d=1$)	-7.069	<0.0001	0.056	0.10	Stationary
Seasonal difference ($D=1, s=52$)	-4.295	0.0005	-	-	Stationary
First + seasonal difference	-7.278	<0.0001	-	-	Stationary

Table 1. Stationarity test results for the weekly load series.

The differenced series behave much as the textbooks predict: both the ordinary first difference and the combined first-and-seasonal difference pass ADF comfortably ($p < 0.001$) and are not rejected by KPSS. The level-series result is a different story — ADF actually rejects the unit-root null at the level ($p = 0.0012$), which on its face would suggest the raw series is already stationary, at odds with the obvious seasonal swings in Figure 1. My reading of this, expanded on in Section 5, is that ADF has limited power against a strongly seasonal alternative: a series cycling regularly around a roughly constant annual mean can look locally mean-reverting to the test even when it isn't stationary in the sense that matters for forecasting. I therefore didn't lean on the level-series ADF result on its own — the level series' autocorrelation function keeps a persistent, significant spike at lag 52 that a first difference alone doesn't remove, and this, together with the STL seasonal strength of 0.871, was the main justification for the seasonal difference used in the SARIMA specification.

3. Methodology

Four families of model were fitted and evaluated on the same held-out two-year (104-week) test period, 14 October 2018 to 4 October 2020, using a chronological train/test split. The feature-based models were assessed with a rolling one-week-ahead evaluation, while the statistical and LSTM models were assessed across the full forecast horizon.

3.1 Benchmark forecasts

Mean, naive, seasonal-naive and drift forecasts (Hyndman and Athanasopoulos, 2021) were included not as serious competitors, but as a bar that any more elaborate model needs to clear to justify its extra cost. Seasonal naive in particular — simply repeating the value from the same week a year earlier — is a demanding benchmark given a seasonal strength as high as 0.871, and, as Section 4 shows, that expectation held up.

3.2 SARIMA and SARIMAX

SARIMA was chosen over plain ARIMA specifically because of the seasonal-strength finding in Section 2.1: a seasonal component this dominant can't be captured by a non-seasonal model without an unwieldy number of ordinary AR/MA terms. Orders were chosen via grid search over $p \in [0,6]$, $d \in [0,2]$, $q \in [0,6]$, with the seasonal difference fixed at $D = 1$ (justified in Section 2.2) and a restricted seasonal search over $(P, Q) \in \{0,1\}^2$, minimising AIC. A full six-dimensional seasonal-and-non-seasonal grid wasn't computationally realistic — even the restricted search of 588 candidate models took just under five hours to fit, since each candidate involves a Kalman-filter state space with a 52-period seasonal component. SARIMAX builds on the selected SARIMA order by adding weekly temperature, heating-degree-days and cooling-degree-days as exogenous regressors, to check whether a known external driver of demand improves on the purely autoregressive model.

3.3 Feature-based regression

A supervised-learning table was built from lagged load (1, 2, 4, 8, 13, 26 and 52 weeks), rolling means and standard deviations of load, lagged temperature, and Fourier/calendar terms for week-of-year, all constructed with shift operations so that no feature for week t draws on information from week t or later. Random Forest (Breiman, 2001) and Histogram Gradient Boosting (Friedman, 2001) were chosen because tree ensembles pick up non-linear interactions — temperature affecting load differently in winter versus summer, for instance — without needing those interactions to be specified explicitly, unlike SARIMAX, and because their feature importances give at least a rough sense of what's driving the forecast.

3.4 LSTM

An LSTM (Hochreiter and Schmidhuber, 1997) was applied to the hourly series, motivated by its established track record in short-term load forecasting (Kong et al., 2019), where it can in principle learn daily and weekly structure directly from a raw sequence without hand-built lag features. A one-week (168-hour) sliding window was used to predict the next hour; three architectures (varying in hidden units, depth and dropout) were compared on a validation split carved out of the training period, and the best-performing configuration (32 units, one layer, no dropout) was retrained with early stopping. To reach the full two-year test horizon, the model was rolled forward iteratively — each hourly prediction fed back in as part of the input for the next step — since it had only been trained to predict one step ahead.

4. Results

4.1 Benchmark forecasts

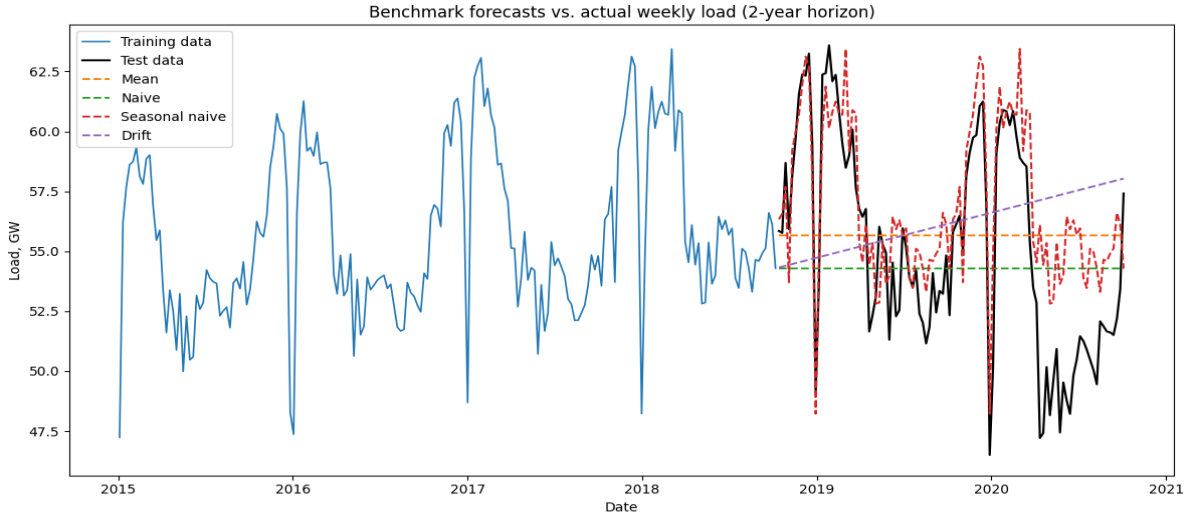


Figure 4. Benchmark forecasts against the two-year held-out test period.

Seasonal naive (MASE 1.732) clearly beat mean (2.831), naive (2.827) and drift (3.243), confirming what the seasonal-strength analysis suggested: since most of the systematic variation in the series is the annual cycle itself, a forecast that simply repeats last year's value captures most of what there is to capture, leaving the other three benchmarks, which ignore seasonality altogether, well behind.

4.2 SARIMA

The AIC-minimising order from the restricted grid search was SARIMA(2,1,6)×(0,1,1,52), with AIC = 300.94. The fitted model, however, came with a convergence warning, and all six of its MA coefficients had standard errors several times larger than the coefficient estimates themselves (e.g. $ma.L1 = 0.930$, $SE = 6.611$, $p = 0.888$) — a strong signal that the model is over-parameterised for a training sample of just 197 weekly observations, and that AIC was rewarding a marginal in-sample fit improvement rather than genuinely useful structure.

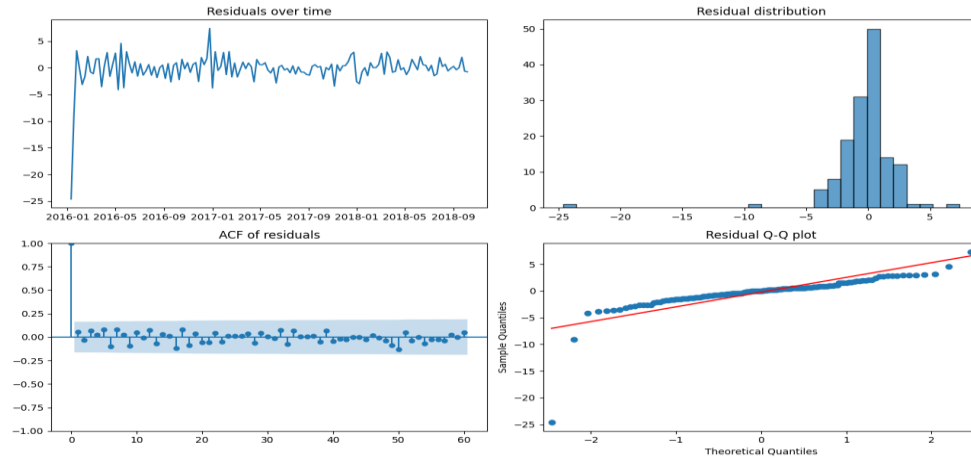


Figure 5. SARIMA residual diagnostics: residuals over time, distribution, ACF, and Q-Q plot.

Despite those estimation concerns, the residual diagnostics in Figure 5 look genuinely clean: the Ljung-Box test found no significant autocorrelation at lags 10, 20 or 52 ($p = 0.757, 0.846, 0.997$), and the Jarque-Bera test did not reject normality ($p = 0.68$). Taken in isolation, this diagnostic picture would normally read as evidence of a well-specified model. The gap between "residuals look like white noise on the training data" and "the model forecasts well out of sample" turned out to be the central lesson of this section.

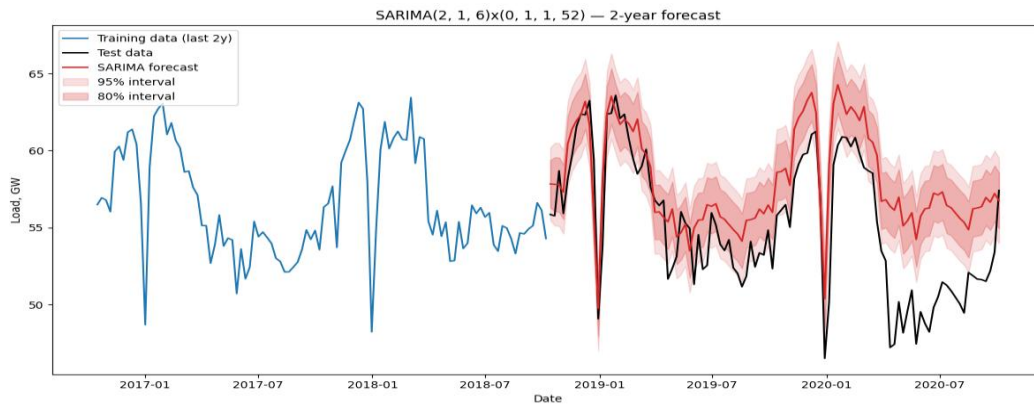


Figure 6. SARIMA two-year forecast with 80% and 95% confidence intervals, against actual load.

On the test period, SARIMA achieved a MASE of 2.301 — worse than seasonal naive — with a mean positive bias of 2.800 GW, meaning it systematically over-predicted load across the forecast horizon (visible in Figure 6, where the forecast sits above the actual series for most of the window). This lines up with the over-parameterisation concern raised above: a flexible model fit tightly to 2015–2018 patterns extrapolated those patterns — including, most likely, a growth trend that didn't continue — into a test period that grew more slowly than the training years and, toward the end, included the COVID-19 demand shock that no model trained on pre-2020 data could have seen coming.

4.3 SARIMAX with temperature

Weekly load correlates with weekly mean Berlin temperature at $r = -0.636$ and with heating degree-days at $r = +0.687$, both in the direction you'd expect for a heating-dominated demand profile. Adding temperature, heating- and cooling-degree-days as exogenous regressors to the same SARIMA order improved MASE from 2.301 to 2.146 and cut the mean bias from 2.800 to 2.591 GW.

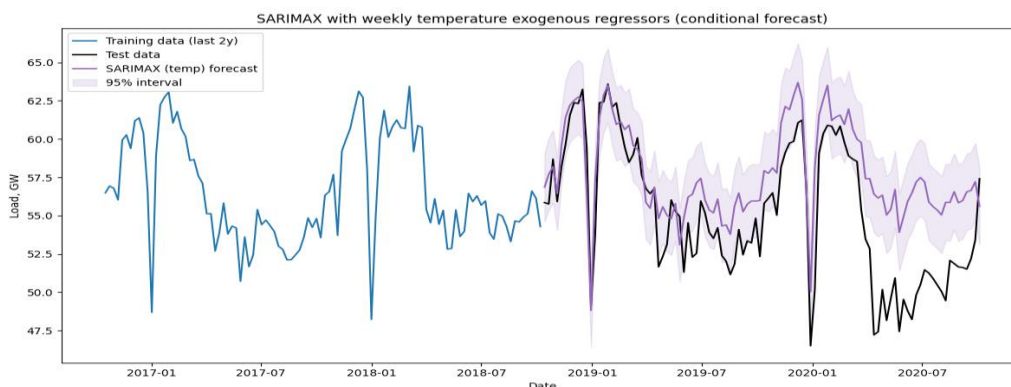


Figure 7. SARIMAX (temperature-conditioned) two-year forecast.

The improvement is real but modest, and SARIMAX still fell short of seasonal naive. Because the same over-parameterised $(2,1,6) \times (0,1,1,52)$ order sits underneath this model, the extra explanatory information from temperature was working on top of an already-biased base forecast rather than fixing the source of that bias. It's also worth restating that this is a conditional, not an operational, forecast: it uses the temperature that actually occurred during the test period rather than a forecast of it, so the reported accuracy overstates what would be achievable in real time.

4.4 Feature-based regression

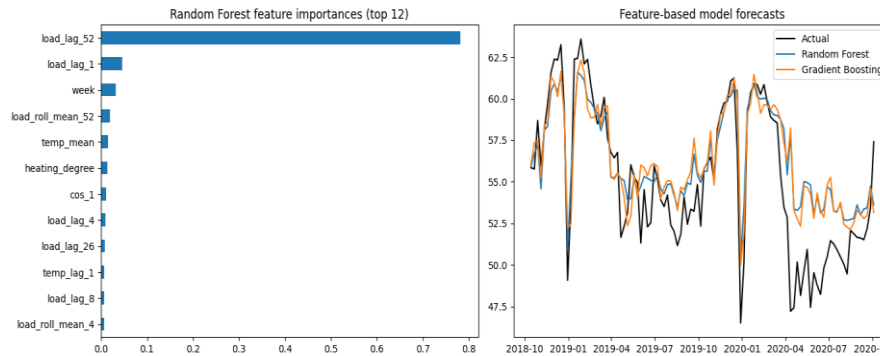


Figure 8. Random Forest feature importances (left) and forecasts against actual load (right).

Random Forest (MASE 1.405) and Gradient Boosting (MASE 1.411) were the only two models to beat seasonal naive, and by a clear margin. The feature-importance ranking in Figure 8 shows `load_lag_52` as by far the dominant predictor, followed by the shorter lags and the rolling-mean features, with temperature contributing a smaller but non-trivial share. This ranking goes some way toward explaining why these models succeeded where SARIMA didn't: they benefit from an explicit seasonal anchor — functionally similar to what seasonal naive already exploits — while also being able to make small, non-linear corrections around that anchor using recent load levels and weather, without being forced through a single global linear/ARMA parameterisation that has to fit the whole series at once.

4.5 LSTM

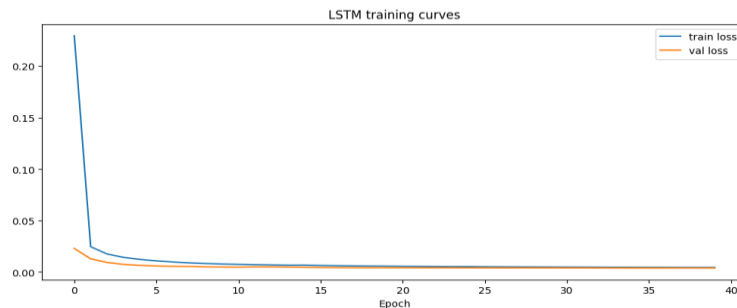


Figure 9. LSTM training and validation loss curves.

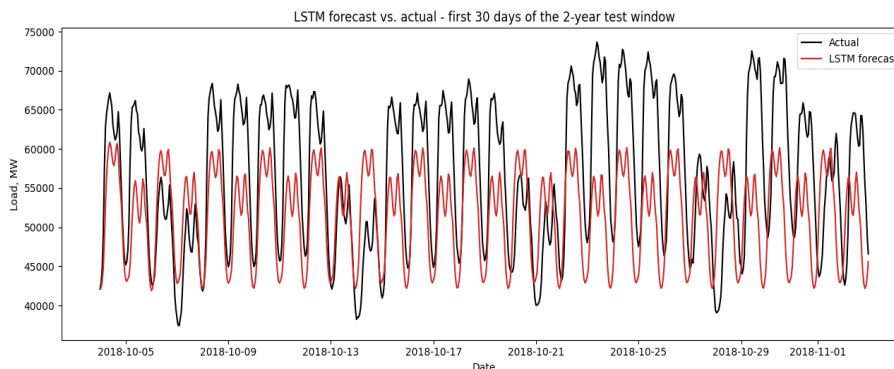


Figure 10. LSTM iterative forecast against actual hourly load, first 30 days of the test window.

Training itself behaved well — Figure 9 shows validation loss tracking training loss with no obvious overfitting within the training period. The forecast, however, turned out to be the weakest of any model tested once aggregated to weekly resolution: MASE 3.319, roughly 1.9 times seasonal naive's error, and marginally behind even the Drift benchmark (MASE 3.243). Figure 10 shows why: over the first 30 days of the two-year test window, the iteratively rolled-forward forecast tracks the broad level of actual load reasonably well at first but gradually loses the amplitude of the daily cycle, settling into a damped oscillation that under-predicts the peaks and drifts away from the true intraday pattern. Because each hourly prediction feeds back in as an input for the next step, small errors compound across the full 17,472-hour horizon, and by

the later stretches of the window the model is effectively forecasting from its own smoothed, accumulated drift rather than from real information about the system.

4.6 Summary of results

Model	MAE (GW)	RMSE (GW)	MASE	Bias (GW)
Random Forest	1.938	2.611	1.405	1.134
Gradient Boosting	1.945	2.621	1.411	1.139
Seasonal Naive	2.319	3.007	1.732	1.732
SARIMAX (temperature, conditional)	2.873	3.601	2.146	2.591
SARIMA	3.080	3.763	2.301	2.800
Naive	3.783	4.459	2.827	-0.882
Mean	3.789	4.397	2.831	0.481
Drift	4.340	5.118	3.243	1.007
LSTM (hourly, aggregated to weekly)	4.441	5.589	3.319	-3.484

Table 2. Evaluation metrics for all models, ranked by MASE (lower is better). MASE is scaled by the in-sample seasonal-naive error, so values below 1.0 would indicate an improvement over the training-period seasonal-naive benchmark; values below seasonal naive's own test-period MASE (1.732) indicate a genuine out-of-sample improvement over it.

5. Critical Discussion

5.1 Why the results turned out the way they did

Two results ran against my expectations and are worth summarising here, with the full detail left to Section 6 (Q1 and Q5) to avoid repeating myself. First, SARIMA and SARIMAX underperformed seasonal naive because the AIC-selected order (six MA terms, a convergence warning, several insignificant coefficients) was over-parameterised for a 197-observation training sample — AIC was rewarding a marginal in-sample fit improvement that didn't generalise. Second, the LSTM's weak showing traces back to the iterative one-step rollout used to reach the two-year horizon: prediction errors compound across roughly 17,500 sequential hourly steps, which is a methodological and architectural issue rather than evidence that LSTMs are unsuited to load forecasting in general. By contrast, Random Forest and Gradient Boosting succeeded largely because their dominant feature, `load_lag_52`, functions much like seasonal naive itself, while the additional lags, rolling statistics and temperature allow small local corrections on top of that seasonal anchor.

5.2 Forecasts against the data actually observed

The two-year test window (October 2018–October 2020) is real data collected after the modelling period it was held out from, and it includes an event no training data could have anticipated: the COVID-19 demand shock visible as an irregular dip toward the end of Figure 1. Every model's positive bias in Table 2, aside from the plain naive forecast, is consistent with this — models trained on 2015–2018 data extrapolated a level of demand that 2020 then undercut. This looks less like a modelling failure and more like a reminder that no model trained purely on historical patterns can anticipate a structural break; a fair operational evaluation would either exclude 2020 from the test window or treat the resulting error inflation as expected rather than diagnostic.

5.3 Interpretability, complexity and covariate availability

A fuller comparison of interpretability and complexity across SARIMAX, the feature-based models and the LSTM, along with a full discussion of covariate availability, is given directly in response to Q5 and Q4 in Section 6, so it isn't repeated in full here. In brief: SARIMAX's apparent interpretability advantage was undermined in this study by a poorly identified fit; the tree ensembles offered a reasonable middle ground via feature importances; and the LSTM was both the least interpretable and least accurate model tested — a useful reminder against defaulting to the most complex architecture available.

6. Answers to the Set Questions

This section answers the six set questions directly. Fuller supporting evidence for each point is developed earlier in the report and referenced below by section.

Q1. Compare all models against the seasonal-naive benchmark. Which, if any, provide a meaningful improvement?

Table 2 (Section 4.6) shows only two models beating seasonal naive (test MASE 1.732): Random Forest (MASE 1.405, a 19% reduction) and Gradient Boosting (MASE 1.411, an 18% reduction). Every other model — SARIMA, SARIMAX, the remaining benchmarks, and the LSTM — was worse than seasonal naive on this test window, some considerably so. This wasn't the outcome I expected going in; I'd anticipated SARIMAX in particular improving on seasonal naive given its access to both autoregressive structure and a weather covariate. Section 5.1 traces this to the AIC-selected SARIMA order being over-parameterised (six largely insignificant MA terms, a convergence warning), which let it fit the training data

closely without generalising well, whereas the tree ensembles succeeded by leaning on a lag-52 feature functionally similar to seasonal naive itself, combined with smaller corrective signals from recent load and temperature.

Q2. How was data leakage avoided when creating temperature lag features?

Three safeguards were applied throughout to minimise leakage. First, all lagged and rolling-window features were built using $\text{shift}(k)$ with $k \geq 1$, so each feature for week t depended only on information available before that week. Second, the final 104 weeks were reserved as a chronological test set rather than drawn from a random split, so the models never trained on observations occurring after the forecast period begins. Third, the `StandardScaler` used for the hourly LSTM was fitted exclusively on the training data and then applied unchanged to the validation and test sets, so future information couldn't leak into preprocessing. For the feature-based models, the evaluation is better understood as a rolling one-week-ahead forecasting exercise, since realised load from earlier test weeks became available when building lag features for later test weeks, rather than as a strict fixed-origin two-year forecast. In addition, observed contemporaneous temperature was supplied during the test period, which makes the SARIMAX and feature-based results conditional (explanatory) forecasts rather than fully operational ones, since future temperature wouldn't be known exactly at the forecast origin.

Q3. For the SARIMAX model, justify the chosen differencing orders and seasonal period.

$d = 1$ and $D = 1$ at $s = 52$ were chosen mainly from the stationarity evidence in Section 2.2 and the STL decomposition in Section 2.1. The first-differenced series passed both ADF and KPSS cleanly, and the STL seasonal strength of 0.871, together with a persistent autocorrelation spike near lag 52 in the level series, justified an additional seasonal difference at the annual (52-week) period — the natural cycle length for weekly-sampled demand data driven by heating and cooling. The level-series ADF result was ambiguous (it rejected the unit-root null even without differencing), which I judged to reflect limited test power against a strongly seasonal series rather than genuine stationarity, so it wasn't used as the basis for the differencing decision. I'm considerably less confident, by contrast, in the specific autoregressive/moving-average orders ($p = 2$, $q = 6$, seasonal $(P,Q) = (0,1)$) returned by the AIC grid search: as discussed under Q1 and in Section 5.1, this order didn't converge cleanly and contains several statistically insignificant coefficients, so while d and D are justified on stationarity grounds, the ARMA orders built on top of them are better read as an AIC artefact than a fully validated specification.

Q4. Do temperature and holiday covariates improve forecast accuracy, and are they known at the forecast origin?

Temperature provided a modest accuracy improvement, since weekly electricity demand showed a clear relationship with weather, with correlations of $r = -0.636$ for mean temperature and $r = +0.687$ for heating degree-days. Adding these as exogenous regressors in the SARIMAX model reduced MASE from 2.301 to 2.146 and lowered mean forecast bias, indicating that weather explains part of the variation in demand. That said, the improvement wasn't enough to outperform the seasonal naive benchmark, partly because it was built on a SARIMA specification that still carried systematic bias. The temperature values used in the test period were the temperatures actually observed rather than weather forecasts, so the resulting SARIMAX forecasts should be read as conditional or explanatory rather than fully operational. Holiday covariates weren't included in the final fitted models, so their impact on accuracy can't be evaluated directly here. Unlike temperature, though, holiday indicators are known with certainty at the forecast origin and so represent a potentially valuable exogenous variable that could improve long-horizon forecasting performance in future work.

Q5. Compare the interpretability and complexity of SARIMAX, feature-based, and neural network models.

SARIMAX is, in principle, the most interpretable of the three, with coefficients that carry a direct statistical meaning and confidence intervals that fall out of the estimation naturally. But Sections 4.2 and 5.3 show that this interpretability is only as trustworthy as the underlying estimation: a coefficient table dominated by large standard errors and a convergence warning isn't, in practice, very informative, whatever its statistical pedigree. The feature-based models (Random Forest, Gradient Boosting) sit in the middle — feature importances (Figure 8) gave a genuinely useful account of what the model was doing, dominated by the lag-52 feature, but they lack a native uncertainty interval and would need an additional step such as quantile regression forests to produce one. The LSTM was both the least interpretable of the three — its predictions can't be traced back to any individual, meaningful parameter — and, in this study, the least accurate of the nine models tested overall, though its weekly-aggregated error came much closer to the weaker benchmarks (Naive, Mean, Drift) than to Random Forest or Gradient Boosting, which is a useful empirical counterpoint to the assumption that architectural sophistication yields better forecasts by default. On complexity, SARIMAX demanded the most manual specification work (order selection, exogenous variable choice), the feature-based models demanded the most feature engineering, and the LSTM demanded the most computational and hyperparameter-tuning effort, without a matching accuracy payoff here.

Q6. Recommend one model for operational use, and justify the choice using accuracy, uncertainty, interpretability, and maintenance.

I'd recommend Random Forest. On accuracy, it's the strongest model tested (MASE 1.405), and Gradient Boosting (MASE 1.411) is close behind — both clearly and consistently beat seasonal naive. On uncertainty, it's currently a weak point — unlike SARIMAX, it doesn't produce a forecast interval natively — but this is a solvable, well-documented gap (quantile regression forests or residual bootstrapping, Section 7), not a fundamental limitation of the model family. On interpretability, its feature-importance output is less rigorous than a regression coefficient, but considerably more informative in practice than the poorly identified SARIMA/SARIMAX coefficients obtained in this study, and easier to explain to non-technical stakeholders than an LSTM's internal weights. On maintenance, it's cheap and quick to retrain as new weekly data arrives, in sharp contrast to the multi-hour SARIMA grid search or the architecture search, scaling and iterative-rollout machinery the LSTM pipeline requires. SARIMAX remains a useful interpretable cross-check precisely because its assumptions are explicit and auditable, and the LSTM is worth keeping in the toolkit for shorter, higher-resolution forecasting tasks where its capacity to learn intraday structure is more likely to pay off — but for this specific two-year weekly operational forecasting task, Random Forest offers the best balance across the four criteria.

7. Limitations and Future Work

- **Model-order selection and architecture:** repeat the SARIMA/SARIMAX search with BIC alongside AIC, and replace the LSTM's iterative rollout with a direct multi-step or encoder-decoder sequence-to-sequence architecture (Kong et al., 2019), addressing the two main weaknesses identified in Section 6.
- **Covariates:** bring in the prepared-but-unused holiday indicators, and substitute genuine short-range weather forecasts (or seasonal-normal climatology at longer horizons) for the observed Berlin temperature, turning SARIMAX and the feature-based models into genuinely operational forecasts.
- **Uncertainty quantification:** add quantile regression forests or residual bootstrapping to Random Forest/Gradient Boosting so they produce a forecast interval comparable to SARIMAX's, rather than only a point forecast.
- **Cross-validation:** use rolling-origin (walk-forward) evaluation across several two-year windows rather than a single split, to check whether the model ranking in Table 2 holds up or is specific to this test period's unusual demand shock.
- **Alternative model families:** benchmark against Prophet (Taylor and Letham, 2018) and global/cross-series deep-learning forecasters, which have performed well on related energy-demand tasks.

8. Conclusion

Across nine forecasting approaches applied to a two-year weekly forecast horizon for German electricity demand, only Random Forest and Gradient Boosting produced a clear, meaningful improvement over the seasonal-naive benchmark, largely by pairing an explicit seasonal anchor with smaller local corrections from recent load and temperature. SARIMA and SARIMAX, despite their added structure, underperformed seasonal naive, mainly because AIC-based order selection produced an over-parameterised, poorly identified model; and the LSTM, despite well-behaved training, produced the least accurate forecast of all — though only narrowly behind the Drift benchmark — because of error accumulation in its iterative multi-step rollout rather than any inherent unsuitability of recurrent networks for this task. Based on accuracy, robustness and maintenance cost, Random Forest is the model I'd recommend for operational deployment, with the caveat that a proper uncertainty-quantification step, genuine (rather than observed) weather inputs, and holiday covariates should all be added before it's used to inform real operational decisions.

References

- Box, G.E.P., Jenkins, G.M., Reinsel, G.C. and Ljung, G.M. (2015) *Time Series Analysis: Forecasting and Control*. 5th edn. Hoboken, NJ: Wiley.
- Breiman, L. (2001) 'Random Forests', *Machine Learning*, 45(1), pp. 5–32.
- Cleveland, R.B., Cleveland, W.S., McRae, J.E. and Terpenning, I. (1990) 'STL: A Seasonal-Trend Decomposition Procedure Based on Loess', *Journal of Official Statistics*, 6(1), pp. 3–73.
- Friedman, J.H. (2001) 'Greedy Function Approximation: A Gradient Boosting Machine', *Annals of Statistics*, 29(5), pp. 1189–1232.
- Hochreiter, S. and Schmidhuber, J. (1997) 'Long Short-Term Memory', *Neural Computation*, 9(8), pp. 1735–1780.
- Hyndman, R.J. and Athanasopoulos, G. (2021) *Forecasting: Principles and Practice*. 3rd edn. Melbourne: OTexts.
- Kong, W., Dong, Z.Y., Jia, Y., Hill, D.J., Xu, Y. and Zhang, Y. (2019) 'Short-Term Residential Load Forecasting Based on LSTM Recurrent Neural Network', *IEEE Transactions on Smart Grid*, 10(1), pp. 841–851.
- Open-Meteo (2024) Historical Weather API. Available at: <https://open-meteo.com/en/docs/historical-weather-api> (Accessed: 10 July 2026).
- Open Power System Data (2020) Time Series Data Package, version 2020-10-06. Available at: https://data.open-power-system-data.org/time_series/ (Accessed: 10 July 2026).